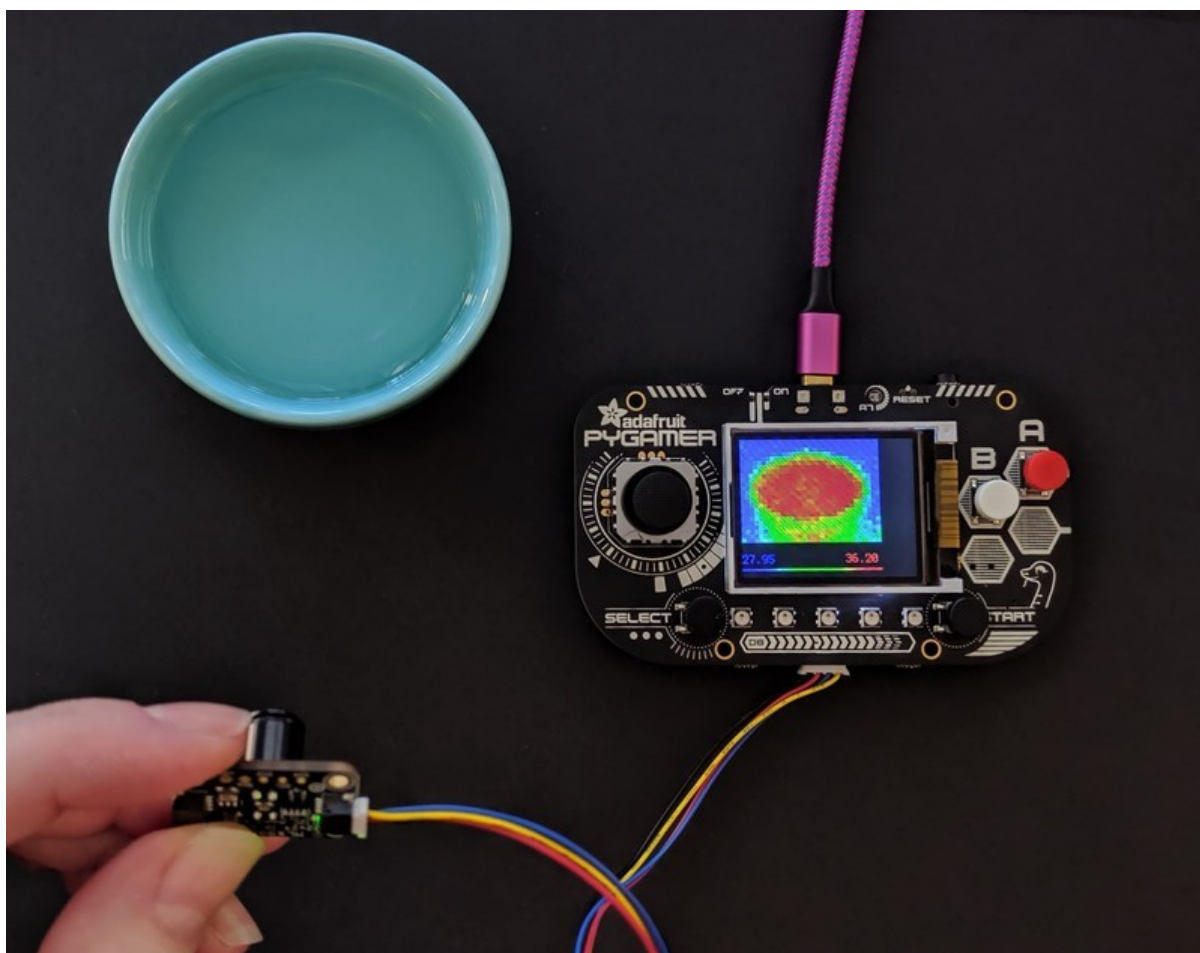




Adafruit MLX90640 IR Thermal Camera

Created by Kattni Rembor



<https://learn.adafruit.com/adafruit-mlx90640-ir-thermal-camera>

Last updated on 2026-08-20 03:17:38 PM UTC

Table of Contents

Overview	5
Pinouts	8
• Power Pins • I2C Logic Pins • STEMMA Connectors	
Arduino Libraries	10
• Library Installation	
Arcada Libraries	11
• Install Libraries • Adafruit Arcada • If you aren't running Arduino IDE 1.8.10 or later, you'll need to install all of the following! • Adafruit NeoPixel • Adafruit FreeTouch • Adafruit Touchscreen • Adafruit SPIFlash • Adafruit Zero DMA • Adafruit GFX • Adafruit ST7735 • Adafruit ILI9341 • Adafruit LIS3DH • Adafruit Sensor • Adafruit ImageReader • ArduinoJson • Adafruit ZeroTimer • Adafruit TinyUSB • Adafruit WavePlayer • SdFat (Adafruit Fork) • Audio - Adafruit Fork	
Arduino Wiring and Example	17
• I2C Wiring • Load Example	
Arduino Docs	21
Arduino PyBadge and PyGamer Thermal Camera	21
• Loading the Example	
Arduino Feather and LCD Thermal Camera	25
• Parts • Wiring • Required Libraries • Code	
Python & CircuitPython	32
• CircuitPython Microcontroller Wiring • Python Computer Wiring • CircuitPython Installation of MLX90640 Library • Python Installation of MLX90640 Library	

- [CircuitPython & Python Usage](#)

[Python Docs](#) 37

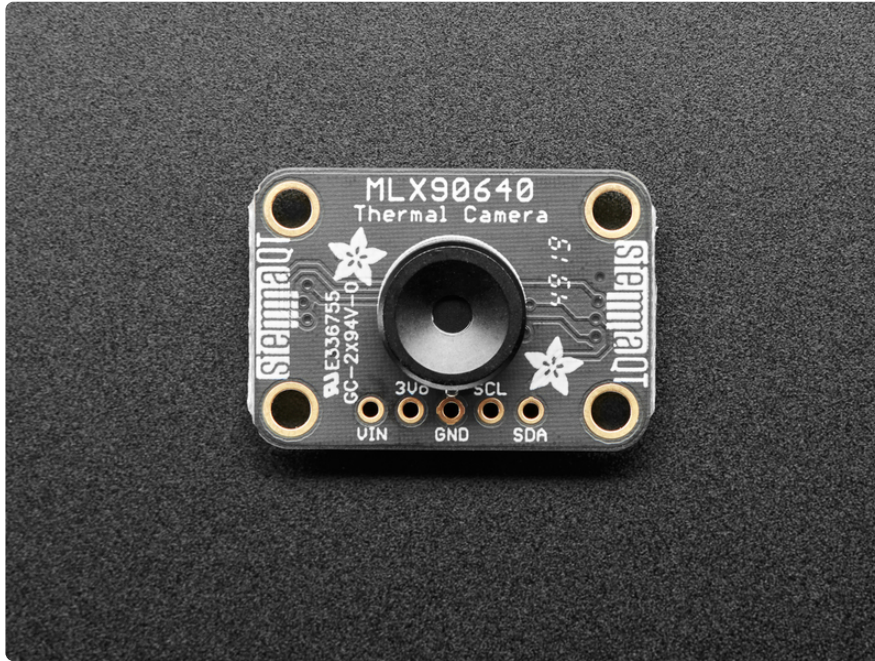
[CircuitPython Thermal Camera](#) 37

- [CircuitPython Microcontroller Wiring](#)
- [CircuitPython Installation of Additional Libraries](#)
- [CircuitPython PyBadge/PyGamer Thermal Camera](#)

[Downloads](#) 42

- [Files](#)
- [Schematic](#)
- [Fab Print](#)

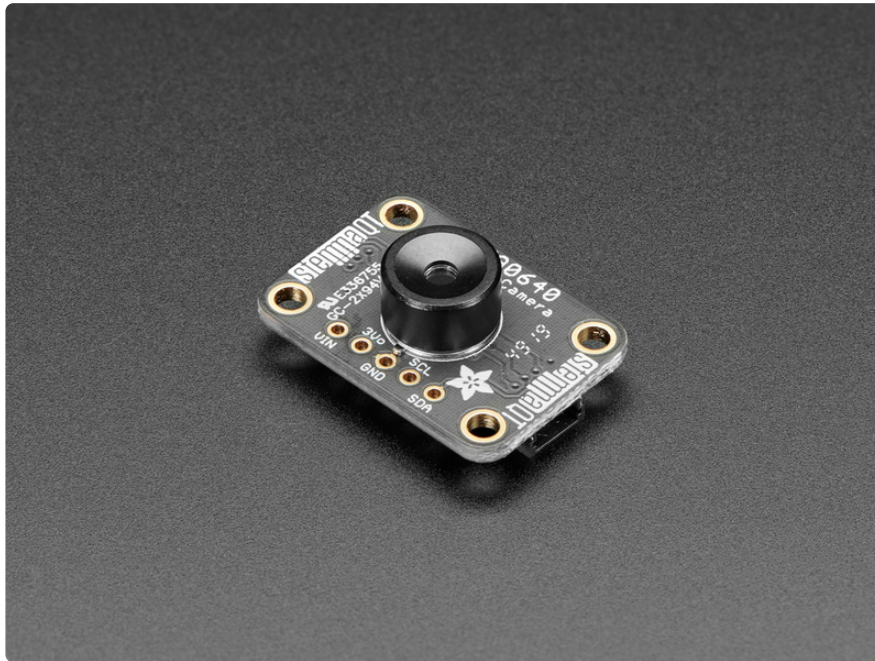
Overview



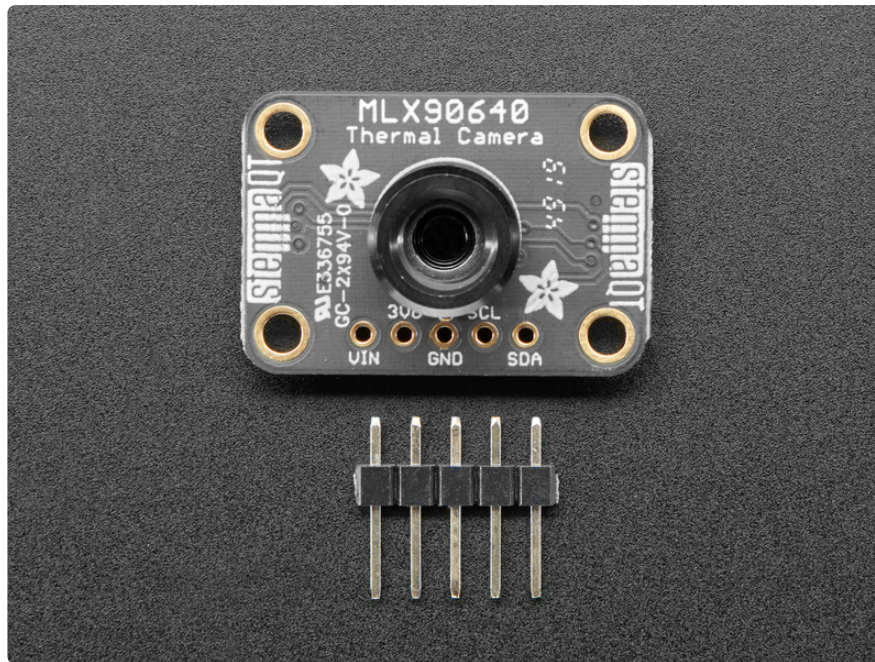
You can now add affordable heat-vision to your project with an Adafruit MLX90640 Thermal Camera Breakout. This sensor contains a 24x32 array of IR thermal sensors. When connected to your microcontroller (or Raspberry Pi) it will return an array of 768 individual infrared temperature readings over I2C. It's like those fancy thermal cameras, but compact and simple enough for easy integration.



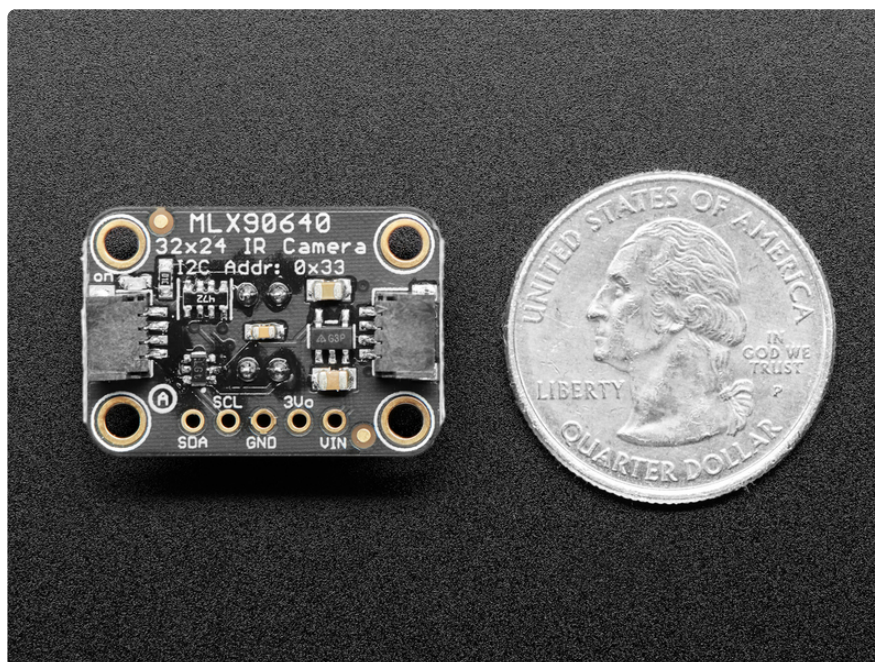
There are two versions: one with a [wider 110°x70° field of view](http://adafru.it/4469) and one with a [narrower 55°x35° field of view](http://adafru.it/4407).



This part will measure temperatures ranging from **-40°C to 300°C** with an accuracy of $\pm 2^{\circ}\text{C}$ (in the 0-100°C range). With a maximum frame rate of 16 Hz (the theoretical limit is 32Hz but we were not able to practically achieve it), It's perfect for creating your own human detector or mini thermal camera. We have code for using this sensor on an Arduino or compatible (the sensor communicates over I2C) or on a Raspberry Pi with Python. If using an Arduino-compatible, you'll need a processor with at least 20KB RAM - a SAMD21 (M0) or SAMD51 (M4) chipset will do nicely. On the Pi, you can even perform interpolation processing with help from the SciPy python library and get some pretty nice results!



This sensor reads the data twice per frame, in a checker-board pattern, so it's normal to see a checker-board dither effect when moving the sensor around - the effect isn't noticeable when things move slowly.

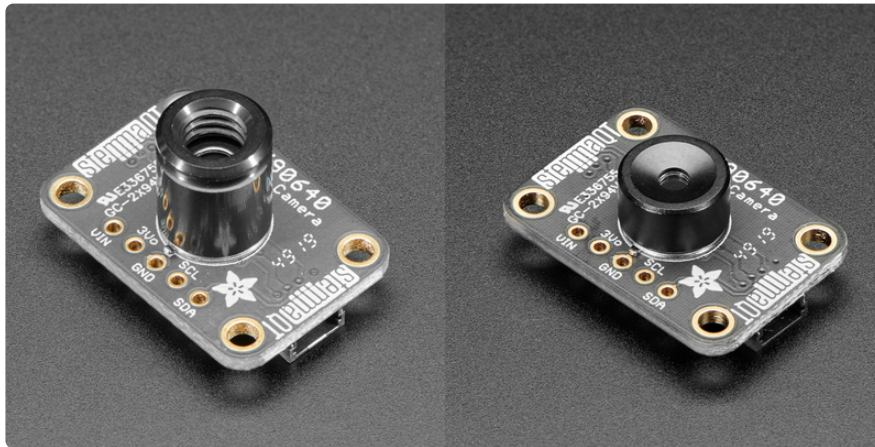


To make it easy to use, we hand-soldered it on a breakout board with a 3.3V regulator and level shifting. So you can use it with any 3V or 5V microcontroller or computer. We've even included [SparkFun qwiic \(https://adafru.it/Fpw\)](https://adafru.it/Fpw) compatible [STEMMA](#)

[QT \(https://adafru.it/Ft4\)](https://adafru.it/Ft4) connectors for the I2C bus so **you don't even need to solder!** Just plug-n-play with any of our STEMMA QT (JST SH) cables.

Even better - We've done all the hard work here, with example code and supporting software libraries to get you up in running in just [a few lines of Arduino \(https://adafru.it/IDj\)](https://adafru.it/IDj) or [Python code \(https://adafru.it/IBv\)](https://adafru.it/IBv).

Pinouts



There is no difference between the breakout for the 55°x35° field of view and the 110°x70° field of view thermal cameras. There is also no difference in the code used with either thermal camera. **The only difference is the length of the lens.** The only way to tell which thermal camera breakout you have is to identify the length of the lens. The longer lens, shown above on the left, is on the 55°x35° field of view thermal camera and is approximately 11.25mm long. The shorter lens, shown above on the right, is on the 110°x70° field of view thermal camera and is approximately 5.7mm long.

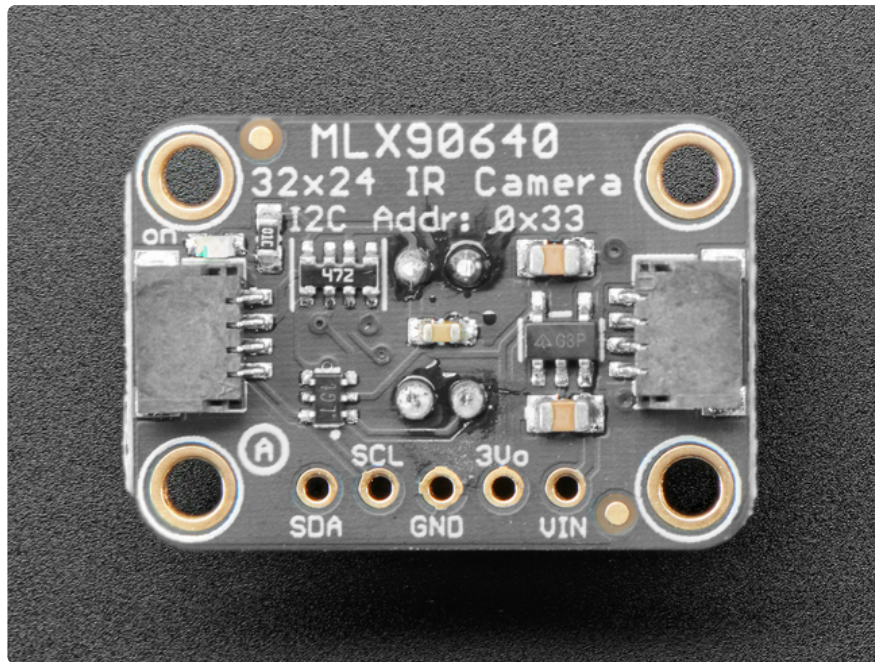


Power Pins

- **VIN** - this is the power pin. Since the sensor chip uses 3 VDC, we have included a voltage regulator on board that will take 3-5VDC and safely convert it down. To power the board, give it the same power as the logic level of your microcontroller - e.g. for a 5V microcontroller like Arduino, use 5V
- **3V** - this is the 3.3V output from the voltage regulator, you can grab up to 100mA from this if you like
- **GND** - common ground for power and logic

I2C Logic Pins

- **SCL** - I2C clock pin, connect to your microcontroller I2C clock line. This pin is level shifted so you can use 3-5V logic, and there's a **4.7K pullup** on this pin.
- **SDA** - I2C data pin, connect to your microcontroller I2C data line. This pin is level shifted so you can use 3-5V logic, and there's a **4.7K pullup** on this pin.



STEMMA Connectors

- [STEMMA QT \(https://adafru.it/Ft4\)](https://adafru.it/Ft4) - These connectors on the back of this breakout allow you to connect to dev boards with **STEMMA QT** connectors or to other things with [various associated accessories \(https://adafru.it/Ft6\)](https://adafru.it/Ft6)

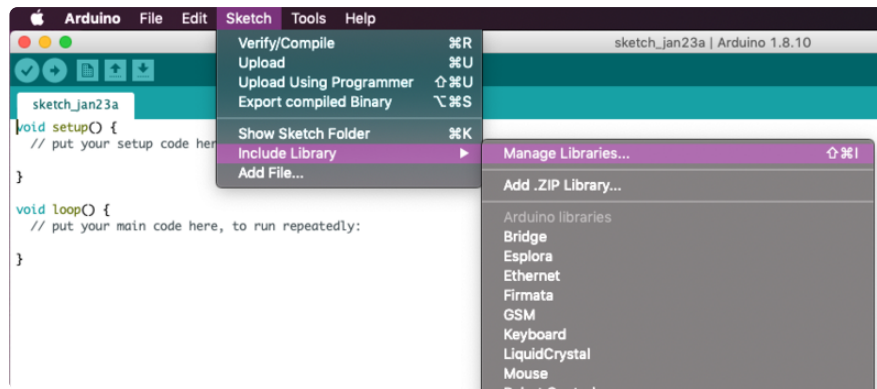
Arduino Libraries

You're going to need to install a few libraries to use this sensor. These are all in the Arduino Library Manager, so they're quite easy to install.

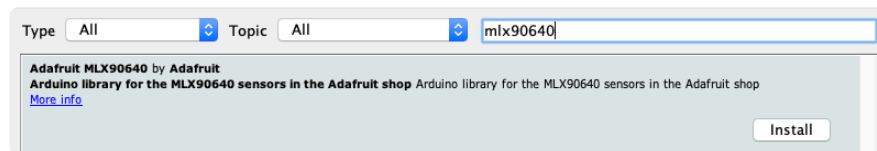
Library Installation

You're going to need to install a few libraries to use this sensor. These are all in the Arduino Library Manager, so they're quite easy to install.

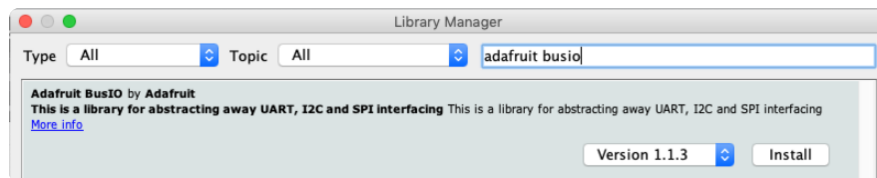
You can install the **Adafruit MLX90640 Library** for Arduino using the Library Manager in the Arduino IDE.



Click the **Manage Libraries...** menu item, search for **Adafruit MLX90640**, and select the **Adafruit MLX90640** library:



Then follow the same process for the **Adafruit BusIO** library.



In the next page, you'll install all the libraries used for Arcada, which is another dependency.

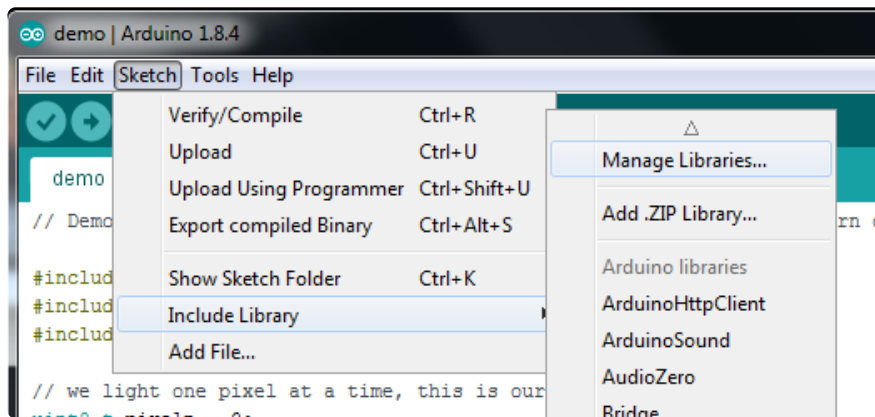
Arcada Libraries

OK now that you have Arduino IDE set up, drivers installed if necessary and you've practiced uploading code, you can start installing all the Libraries we'll be using to program it.

There's a lot of libraries!

Install Libraries

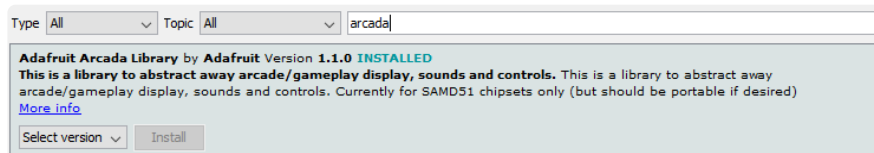
Open up the library manager...



And install the following libraries:

Adafruit Arcada

This library generalizes the hardware for you so you can read the joystick, draw to the display, read files, etc. without having to worry about the underlying methods



Text

emphasized

with a

yellow

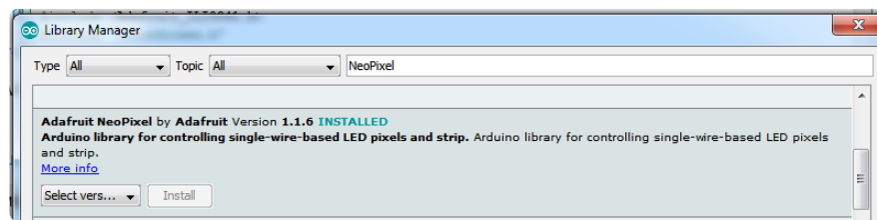
exclamation:

If you use Arduino 1.8.10 or later, the IDE will automatically install all the libraries you need to run all the Arcada demos when you install Arcada. We strongly recommend using the latest IDE so you don't miss one of the libraries!

If you aren't running Arduino IDE 1.8.10 or later, you'll need to install all of the following!

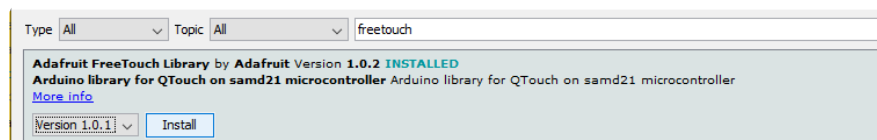
Adafruit NeoPixel

This will let you light up the status LEDs on the front/back



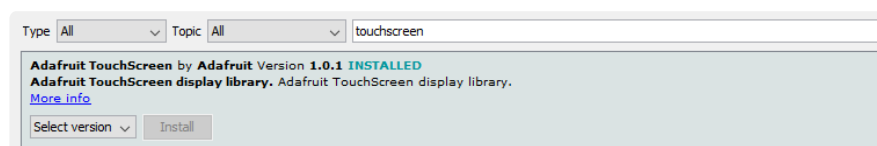
Adafruit FreeTouch

This is the open source version of QTouch for SAMD21 boards



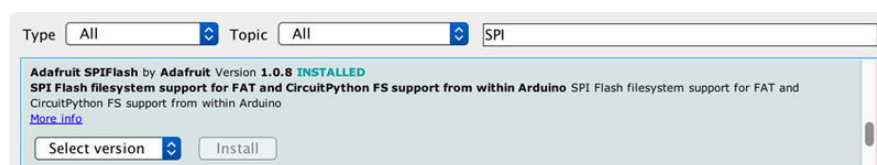
Adafruit Touchscreen

Used by Adafruit Arcada for touchscreen input (required even if your Arcada board does not have a touchscreen)



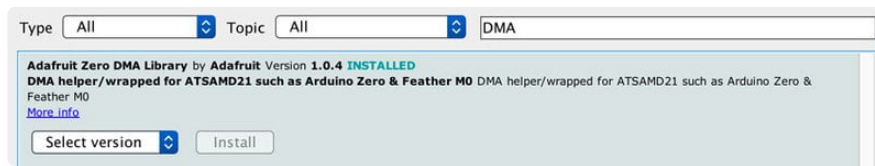
Adafruit SPIFlash

This will let you read/write to the onboard FLASH memory with super-fast QSPI support



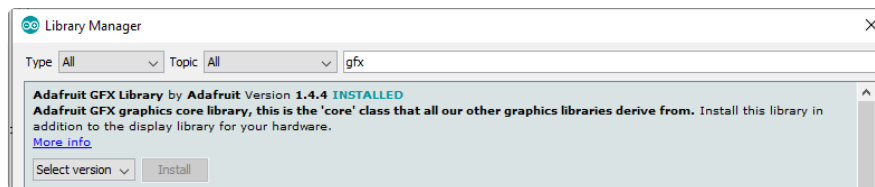
Adafruit Zero DMA

This is used by the Graphics Library if you choose to use DMA



Adafruit GFX

This is the graphics library used to draw to the screen



If using an older (pre-1.8.10) Arduino IDE, locate and install **Adafruit_BusIO** (newer versions do this one automatically).

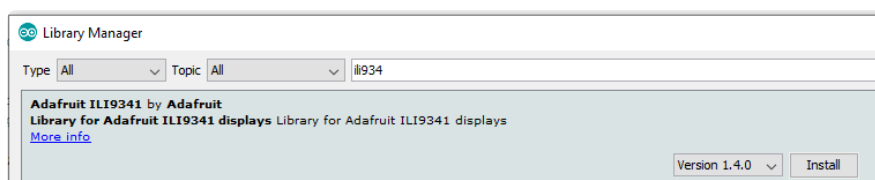
Adafruit ST7735

The display on the PyBadge/PyGamer & other Arcada boards



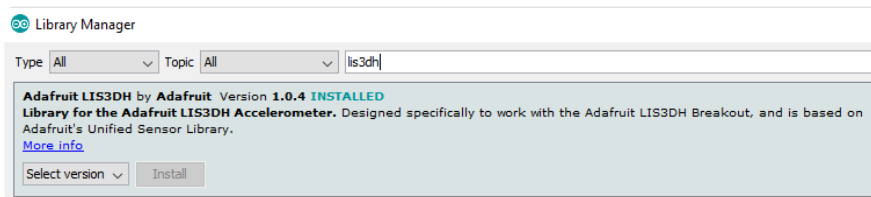
Adafruit ILI9341

The display on the PyPortal & other Arcada boards



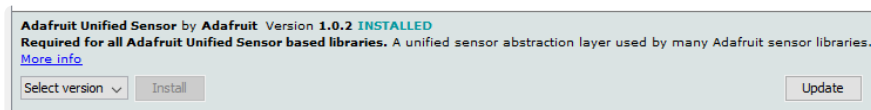
Adafruit LIS3DH

For reading the accelerometer data, required even if one is not on the board



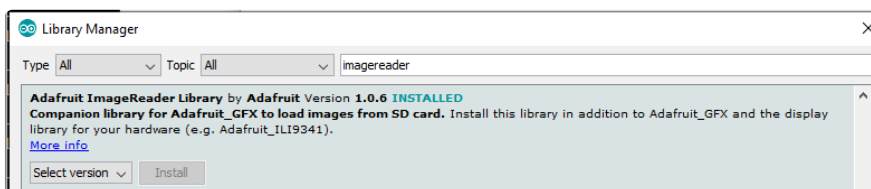
Adafruit Sensor

Needed by the LIS3DH Library, required even if one is not on the board



Adafruit ImageReader

For reading bitmaps from SPI Flash or SD and displaying



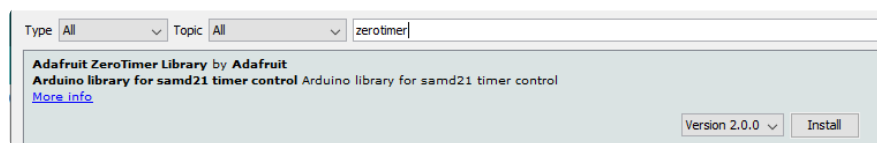
ArduinoJson

We use this library to read and write configuration files



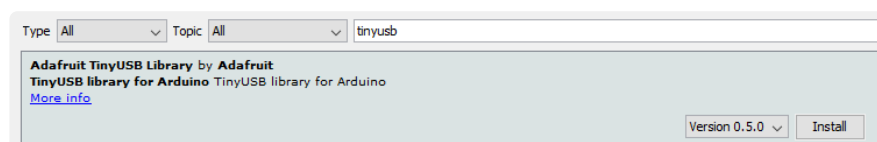
Adafruit ZeroTimer

We use this library to easily set timers and callbacks on the SAMD processors



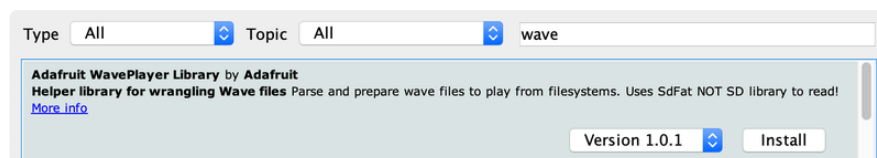
Adafruit TinyUSB

This lets us do cool stuff with USB like show up as a Keyboard or Disk Drive



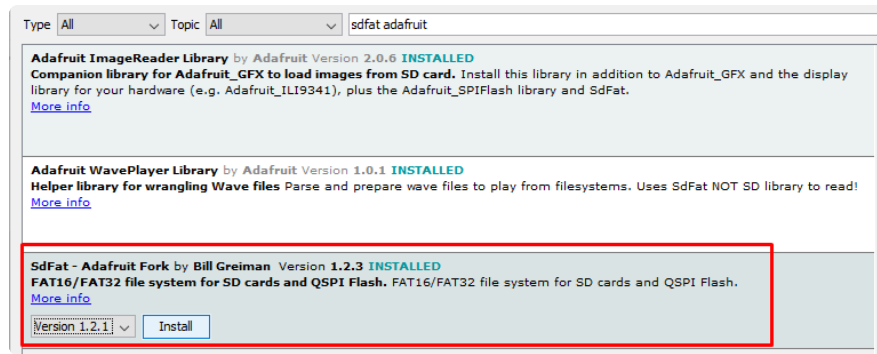
Adafruit WavePlayer

Helps us play .WAV sound files.



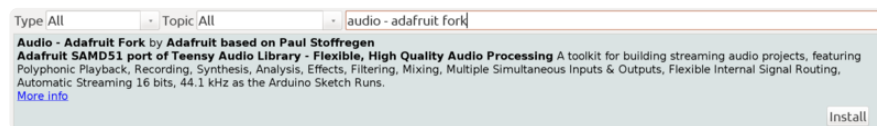
SdFat (Adafruit Fork)

The Adafruit fork of the really excellent SD card library that gives a lot more capability than the default SD library



Audio - Adafruit Fork

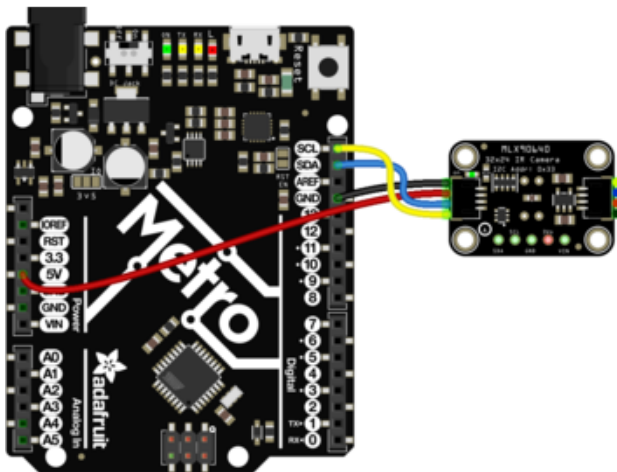
Our fork of the Audio library provides a toolkit for building streaming audio projects.



Arduino Wiring and Example

I2C Wiring

Use this wiring to connect via I2C interface.



Connect board **VIN** (red wire) to **Arduino 5V** if you are running a **5V** board Arduino (Uno, etc.). If your board is **3V**, connect to that instead.

Connect board **GND** (black wire) to **Arduino GND**

Connect board **SCL** (yellow wire) to **Arduino SCL**

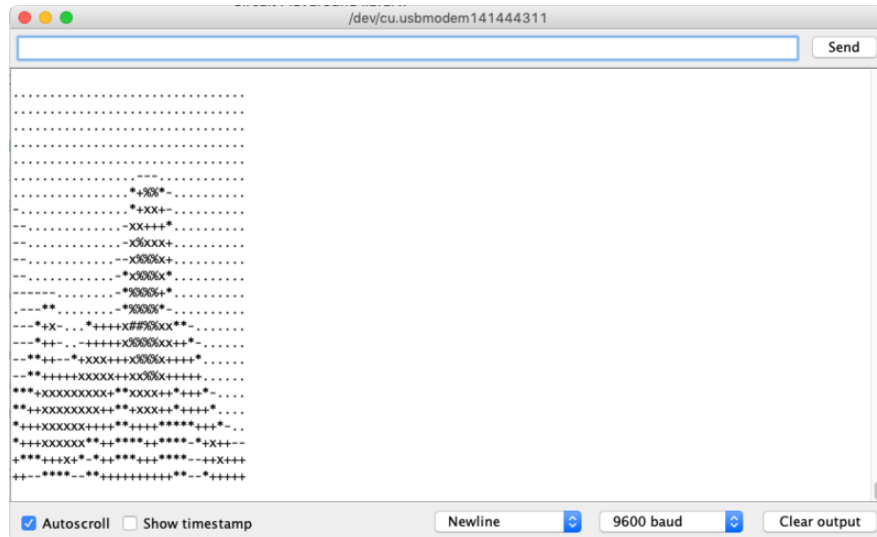
Connect board **SDA** (blue wire) to **Arduino SDA**

The final results should resemble the illustration above, showing an Adafruit Metro development board.

Load Example

Open up **File -> Examples -> Adafruit MLX90640 -> MLX90640_simpletest** and upload to your Arduino wired up to the sensor.

Once you upload the code and open the Serial Monitor (**Tools->Serial Monitor**) at **115200** baud, you will see an ASCII representation of the thermal camera printed. You should see something similar to this:

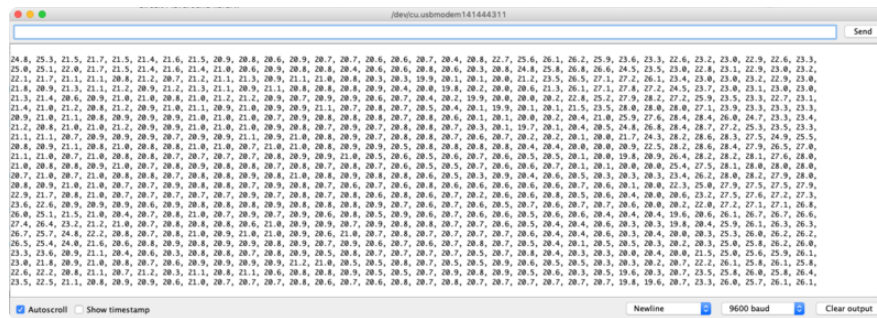


Point the thermal camera at objects of differing temperatures to see the ASCII image change!

You also have the option to see the temperatures printed out in a grid. Change the lines below **// uncomment *one* of the below** to comment out the **ASCIIART** line and uncomment the **TEMPERATURES** line so that they match below:

```
1 // uncomment *one* of the below
2 #define PRINT_TEMPERATURES
3 //#define PRINT_ASCIIART
```

Upload the code and open the Serial Monitor (**Tools->Serial Monitor**) at **115200** baud, and you will see a grid of temperatures in Celsius printed. To fit the entire grid, you may need to resize the serial monitor window.



Point the thermal camera at objects of differing temperatures to see the printed temperatures change.

```

1  #include <Adafruit_MLX90640.h>
2
3  Adafruit_MLX90640 mlx;
4  float frame[32*24]; // buffer for full frame of temperatures
5
6  // uncomment *one* of the below
7  // #define PRINT_TEMPERATURES
8  #define PRINT_ASCIIART
9
10 void setup() {
11   while (!Serial) delay(10);
12   Serial.begin(115200);
13   delay(100);
14
15   Serial.println("Adafruit MLX90640 Simple Test");
16   if (! mlx.begin(MLX90640_I2CADDR_DEFAULT, &Wire)) {
17     Serial.println("MLX90640 not found!");
18     while (1) delay(10);
19   }
20   Serial.println("Found Adafruit MLX90640");
21
22   Serial.print("Serial number: ");
23   Serial.print(mlx.serialNumber[0], HEX);
24   Serial.print(mlx.serialNumber[1], HEX);
25   Serial.println(mlx.serialNumber[2], HEX);
26
27   //mlx.setMode(MLX90640_INTERLEAVED);
28   mlx.setMode(MLX90640_CHESS);
29   Serial.print("Current mode: ");
30   if (mlx.getMode() == MLX90640_CHESS) {
31     Serial.println("Chess");
32   } else {
33     Serial.println("Interleave");
34   }
35
36   mlx.setResolution(MLX90640_ADC_18BIT);
37   Serial.print("Current resolution: ");
38   mlx90640_resolution_t res = mlx.getResolution();
39   switch (res) {
40     case MLX90640_ADC_16BIT: Serial.println("16 bit"); break;
41     case MLX90640_ADC_17BIT: Serial.println("17 bit"); break;

```

```

42     case MLX90640_ADC_18BIT: Serial.println("18 bit"); break;
43     case MLX90640_ADC_19BIT: Serial.println("19 bit"); break;
44 }
45
46 mlx.setRefreshRate(MLX90640_2_HZ);
47 Serial.print("Current frame rate: ");
48 mlx90640_refreshrate_t rate = mlx.getRefreshRate();
49 switch (rate) {
50     case MLX90640_0_5_HZ: Serial.println("0.5 Hz"); break;
51     case MLX90640_1_HZ: Serial.println("1 Hz"); break;
52     case MLX90640_2_HZ: Serial.println("2 Hz"); break;
53     case MLX90640_4_HZ: Serial.println("4 Hz"); break;
54     case MLX90640_8_HZ: Serial.println("8 Hz"); break;
55     case MLX90640_16_HZ: Serial.println("16 Hz"); break;
56     case MLX90640_32_HZ: Serial.println("32 Hz"); break;
57     case MLX90640_64_HZ: Serial.println("64 Hz"); break;
58 }
59 }
60
61 void loop() {
62     delay(500);
63     if (mlx.getFrame(frame) != 0) {
64         Serial.println("Failed");
65         return;
66     }
67     Serial.println("=====");
68     Serial.print("Ambient temperature = ");
69     Serial.print(mlx.getTa(false)); // false = no new frame capture
70     Serial.println(" degC");
71     Serial.println();
72     Serial.println();
73     for (uint8_t h=0; h<24; h++) {
74         for (uint8_t w=0; w<32; w++) {
75             float t = frame[h*32 + w];
76 #ifndef PRINT_TEMPERATURES
77             Serial.print(t, 1);
78             Serial.print(", ");
79 #endif
80 #ifndef PRINT_ASCIIART
81             char c = '&';
82             if (t < 20) c = ' ';
83             else if (t < 23) c = '!';
84             else if (t < 25) c = '-';
85             else if (t < 27) c = '*';
86             else if (t < 29) c = '+';
87             else if (t < 31) c = 'x';
88             else if (t < 33) c = '%';
89             else if (t < 35) c = '#';
90             else if (t < 37) c = 'X';
91             Serial.print(c);
92 #endif
93         }
94         Serial.println();
95     }
96 }

```

https://github.com/adafruit/Adafruit_MLX90640/blob/master/examples/MLX90640_simpletest/MLX90640_simpletest.ino

Arduino Docs

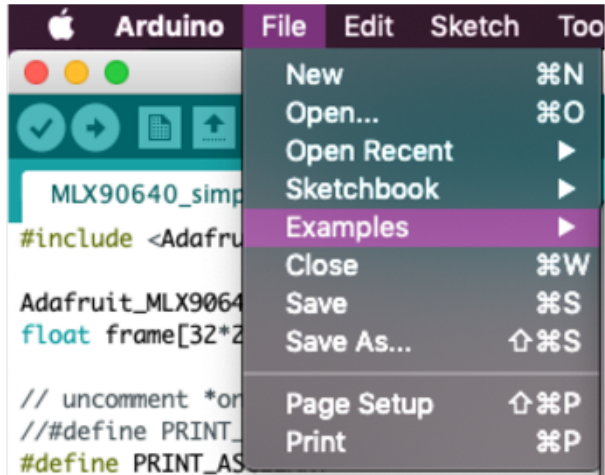
[Arduino Docs \(https://adafru.it/lzA\)](https://adafru.it/lzA)

Arduino PyBadge and PyGamer Thermal Camera

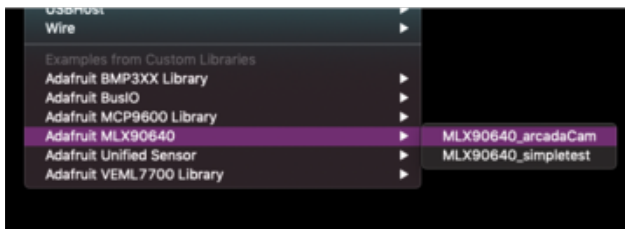
The PyBadge and PyGamer boards both have displays built in, as well as STEMMA I2C connectors. The MLX90640 comes with STEMMA QT/QWIIC connectors to allow for attaching the board easily to projects with no soldering required. Adafruit sells a [STEMMA to STEMMA QT cable \(http://adafru.it/4424\)](http://adafru.it/4424) that allows you to plug this breakout into the STEMMA connector found on a number of Adafruit microcontroller boards, including PyBadge and PyGamer. The following example uses the PyBadge or PyGamer and the MLX90640 to create a super easy-to-assemble thermal camera with a display!

Start by following your board's guide on installing Arduino IDE, and support for the board you have. Then, follow the instructions on [the Arduino page in this guide \(https://adafru.it/IDl\)](https://adafru.it/IDl) to install the base libraries needed for this breakout. Finally, install the [Adafruit Arcada libraries \(https://adafru.it/EUk\)](https://adafru.it/EUk) (there's a lot of em!).

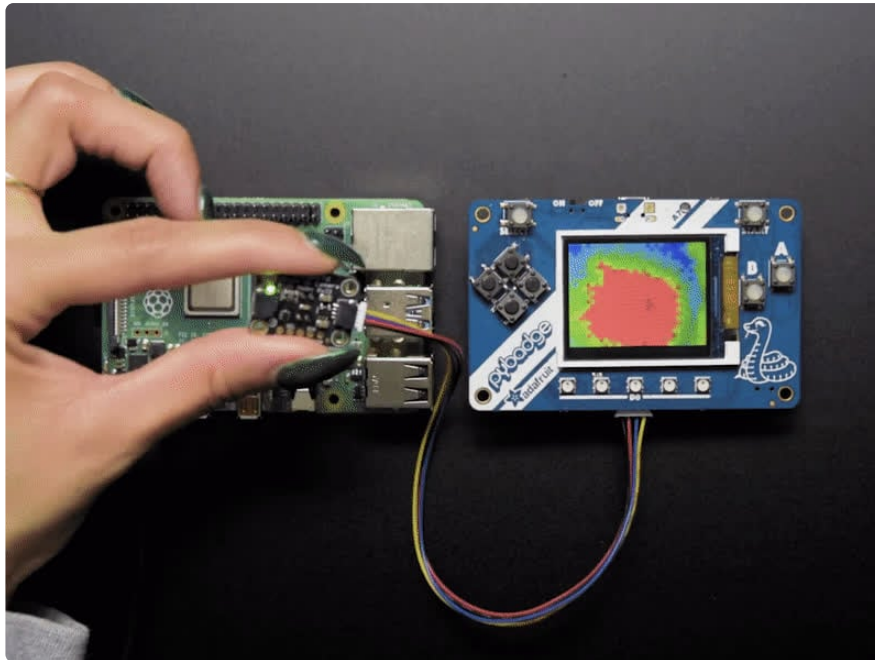
Loading the Example



Go to **File > Examples > Adafruit MLX90640 > MLX90640_arcadacam** to open the ArcadaCam Arduino thermal camera example, and then load it onto your PyBadge or PyGamer.



Now point the thermal camera at various objects to see a heat map displayed on your board!



```

1  #include <Adafruit_MLX90640.h>
2  #include "Adafruit_Arcada.h"
3  Adafruit_Arcada arcada;
4
5
6  Adafruit_MLX90640 mlx;
7  float frame[32*24]; // buffer for full frame of temperatures
8
9  //low range of the sensor (this will be blue on the screen)
10 #define MINTEMP 20
11
12 //high range of the sensor (this will be red on the screen)
13 #define MAXTEMP 35
14
15 //the colors we will be using
16 const uint16_t camColors[] = {0x480F,
17 0x400F,0x400F,0x400F,0x4010,0x3810,0x3810,0x3810,0x3810,0x3010,0x3010,
18 0x3010,0x2810,0x2810,0x2810,0x2810,0x2010,0x2010,0x2010,0x1810,0x1810,
19 0x1811,0x1811,0x1011,0x1011,0x1011,0x0811,0x0811,0x0811,0x0011,0x0011,
20 0x0011,0x0011,0x0011,0x0031,0x0031,0x0051,0x0072,0x0072,0x0092,0x00B2,
21 0x00B2,0x00D2,0x00F2,0x00F2,0x0112,0x0132,0x0152,0x0152,0x0172,0x0192,
22 0x0192,0x01B2,0x01D2,0x01F3,0x01F3,0x0213,0x0233,0x0253,0x0253,0x0273,
23 0x0293,0x02B3,0x02D3,0x02D3,0x02F3,0x0313,0x0333,0x0333,0x0353,0x0373,
24 0x0394,0x03B4,0x03D4,0x03D4,0x03F4,0x0414,0x0434,0x0454,0x0474,0x0474,
25 0x0494,0x04B4,0x04D4,0x04F4,0x0514,0x0534,0x0534,0x0554,0x0554,0x0574,
26 0x0574,0x0573,0x0573,0x0573,0x0572,0x0572,0x0572,0x0571,0x0591,0x0591,
27 0x0590,0x0590,0x058F,0x058F,0x058F,0x058E,0x05AE,0x05AE,0x05AD,0x05AD,
28 0x05AD,0x05AC,0x05AC,0x05AB,0x05CB,0x05CB,0x05CA,0x05CA,0x05CA,0x05C9,
29 0x05C9,0x05C8,0x05E8,0x05E8,0x05E7,0x05E7,0x05E6,0x05E6,0x05E6,0x05E5,
30 0x05E5,0x0604,0x0604,0x0604,0x0603,0x0603,0x0602,0x0602,0x0601,0x0621,
31 0x0621,0x0620,0x0620,0x0620,0x0620,0x0620,0x0E20,0x0E20,0x0E40,0x1640,0x1640,
32 0x1E40,0x1E40,0x2640,0x2640,0x2E40,0x2E60,0x3660,0x3660,0x3E60,0x3E60,
33 0x3E60,0x4660,0x4660,0x4E60,0x4E80,0x5680,0x5680,0x5E80,0x5E80,0x6680,
34 0x6680,0x6E80,0x6EA0,0x76A0,0x76A0,0x7EA0,0x7EA0,0x86A0,0x86A0,0x8EA0,
35 0x8EC0,0x96C0,0x96C0,0x9EC0,0x9EC0,0xA6C0,0xAEC0,0xAEC0,0xB6E0,0xB6E0,
36 0xBEE0,0xBEE0,0xC6E0,0xC6E0,0xC6E0,0xC6E0,0xD6E0,0xD700,0xDF00,0xDEE0,

```

```

37 0xDEC0,0xDEA0,0xDE80,0xDE80,0xE660,0xE640,0xE620,0xE600,0xE5E0,0xE5C0,
38 0xE5A0,0xE580,0xE560,0xE540,0xE520,0xE500,0xE4E0,0xE4C0,0xE4A0,0xE480,
39 0xE460,0xEC40,0xEC20,0xEC00,0xEBE0,0xEBC0,0xEBA0,0xEB80,0xEB60,0xEB40,
40 0xEB20,0xEB00,0xEAE0,0xEAC0,0xEAA0,0xEA80,0xEA60,0xEA40,0xF220,0xF200,
41 0xF1E0,0xF1C0,0xF1A0,0xF180,0xF160,0xF140,0xF100,0xF0E0,0xF0C0,0xF0A0,
42 0xF080,0xF060,0xF040,0xF020,0xF800,};
43
44 uint16_t displayPixelWidth, displayPixelHeight;
45
46 void setup() {
47   if (!arcada.arcadaBegin()) {
48     Serial.print("Failed to begin");
49     while (1);
50   }
51   arcada.displayBegin();
52   // Turn on backlight
53   arcada.setBacklight(255);
54
55   Serial.begin(115200);
56   //while (!Serial);
57
58   arcada.display->fillScreen(ARCADA_BLACK);
59   displayPixelWidth = arcada.display->width() / 32;
60   displayPixelHeight = arcada.display->width() / 32; //Keep pixels square
61
62   delay(100);
63
64   Serial.println("Adafruit MLX90640 Camera");
65   if (! mlx.begin(MLX90640_I2CADDR_DEFAULT, &Wire)) {
66     arcada.haltBox("MLX90640 not found!");
67   }
68   Serial.println("Found Adafruit MLX90640");
69
70   Serial.print("Serial number: ");
71   Serial.print(mlx.serialNumber[0], HEX);
72   Serial.print(mlx.serialNumber[1], HEX);
73   Serial.println(mlx.serialNumber[2], HEX);
74
75   mlx.setMode(MLX90640_CHESS);
76   mlx.setResolution(MLX90640_ADC_18BIT);
77   mlx.setRefreshRate(MLX90640_8_HZ);
78   Wire.setClock(1000000); // max 1 MHz
79 }
80
81
82 void loop() {
83   uint32_t timestamp = millis();
84   if (mlx.getFrame(frame) != 0) {
85     Serial.println("Failed");
86     return;
87   }
88
89   int colorTemp;
90   for (uint8_t h=0; h<24; h++) {
91     for (uint8_t w=0; w<32; w++) {
92       float t = frame[h*32 + w];

```

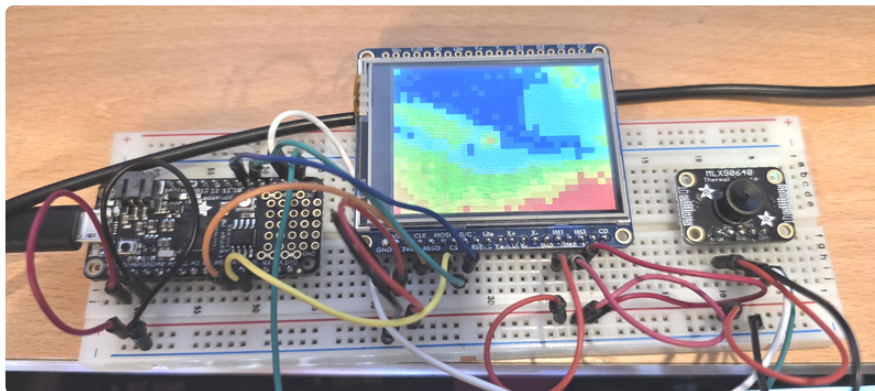
```

93 // Serial.print(t, 1); Serial.print(" ");
94
95 t = min(t, MAXTEMP);
96 t = max(t, MINTEMP);
97
98 uint8_t colorIndex = map(t, MINTEMP, MAXTEMP, 0, 255);
99
100 colorIndex = constrain(colorIndex, 0, 255);
101 //draw the pixels!
102 arcada.display->fillRect(displayPixelWidth * w, displayPixelHeight * h,
103 displayPixelHeight, displayPixelWidth,
104 camColors[colorIndex]);
105 }
106 }
107 Serial.print((millis()-timestamp) / 2); Serial.println(" ms per frame (2 frames per display)");
108 Serial.print(2000.0 / (millis()-timestamp)); Serial.println(" FPS (2 frames per display)");
109 }

```

https://github.com/adafruit/Adafruit_MLX90640/blob/master/examples/MLX90640_arcadaCam/MLX90640_arcadaCam.ino

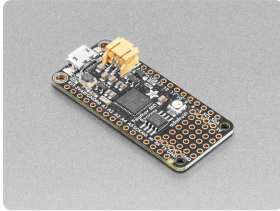
Arduino Feather and LCD Thermal Camera



Here is a version of the thermal camera in Arduino with a Feather and discrete LCD display as an alternate to the Arcada-based devices like PyPortal and PyGamer.

This version uses an Adafruit Feather M0 Express and an ILI9341-based display in addition to the MLX90640 thermal sensor. The display connects over an SPI interface and the sensor over an I2C interface.

Parts

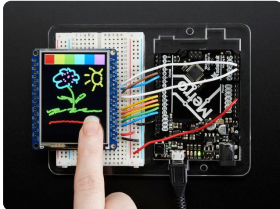


Featured

Adafruit Feather M0 Express

At the Feather M0's heart is an ATSAMD21G18 ARM Cortex M0+ processor, clocked at 48 MHz and at 3.3V logic, the same one used in the new

<https://www.adafruit.com/product/3403>



Featured

Adafruit 2.4" TFT LCD with Touchscreen Breakout w/MicroSD Socket

Add some jazz & pizzazz to your project with a color touchscreen LCD. This TFT display is 2.4" diagonal with a bright (4 white-LED) backlight and it's colorful! 240x320...

<https://www.adafruit.com/product/2478>



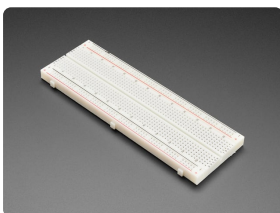
Featured

Adafruit MLX90640 IR Thermal Camera Breakout

You can now add affordable heat-vision to your project and with an Adafruit MLX90640 Thermal Camera Breakout. This sensor contains a 24x32 array of IR thermal sensors. When connected...

<https://www.adafruit.com/product/4407>

The above sensor has a 55 degree field of view. for a 110 degree field of view, use [this version](http://adafru.it/4469) (<http://adafru.it/4469>).

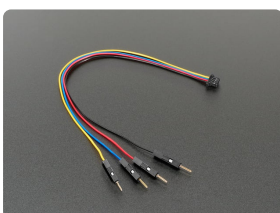


Featured

Full Sized Premium Breadboard - 830 Tie Points

This is a 'full-size' premium quality breadboard, 830 tie points. Good for small and medium projects. It's 2.2" x 7" (5.5 cm x 17 cm) with a standard double-strip...

<https://www.adafruit.com/product/239>

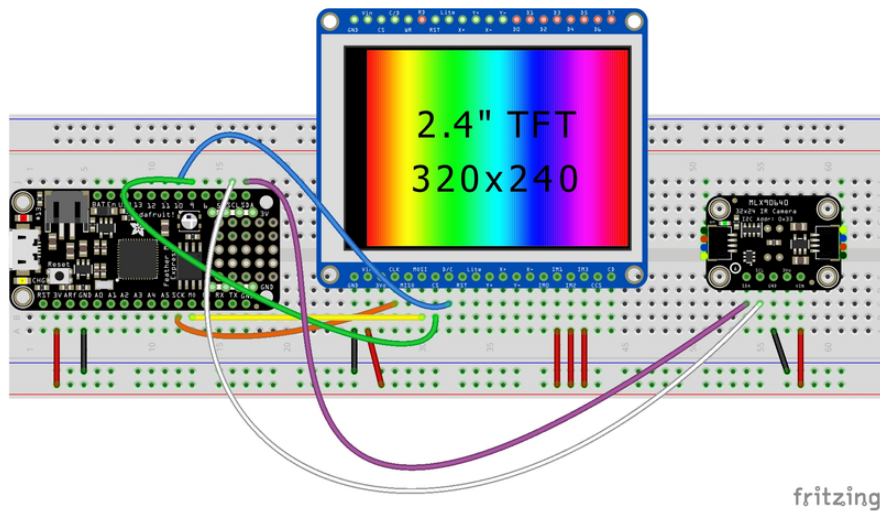


Featured

STEMMA QT / Qwiic JST SH 4-pin to Premium Male Headers Cable

This 4-wire cable is a little over 150mm / 6" long and fitted with JST-SH female 4-pin connectors on one end and premium Dupont male headers on the other. Compared with the...

<https://www.adafruit.com/product/4209>



Wiring

See the Fritzing diagram above. The following connections are needed:

From the Feather to the display:

- Feather 3V pin to display Vin
- Feather GND pin to display GND
- Feather SCK to display CLK
- Feather MO to display MOSI
- Feather Pin 10 to display D/C
- Feather Pin 9 to display CS
- Feather or display 3V output (never 5V!) to IM3, IM2, and IM1 (but not IM0!) pins. This configures the breakout to use its SPI interface. See the breakout guide for [details on soldering closed these connections to make the SPI interface the default](https://adafru.it/dLU) (<https://adafru.it/dLU>).

Connections from the Feather to the sensor:

The sensor has STEMMA QT connectors. This Feather does not. A cable like [STEMMA QT to male pins](http://adafru.it/4209) (<http://adafru.it/4209>) may be used if a Stemma connection is desired.

- Feather 3V to sensor VIN

- Feather GND to sensor GND
- Feather SCL to sensor SCL
- Feather SDA to sensor SDA

NOTE: The MLX90640 board in Fritzing is **UPSIDE DOWN** from the usual placement with the camera sensor up. Use the connection pins noted to get the wiring correct.

For the STEMMA QT cable:

- Red - 3.3VDC Power
- Black - Ground
- Blue - I2C SDA Data
- Yellow - I2C SCL Clock



Required Libraries

This project uses the **Adafruit MLX90640** library, the **Adafruit ILI9341** library, and the **Adafruit GFX** library. Be sure to install all library dependencies and ensure all libraries are at their latest versions. The process is the same as the [Arduino Libraries page \(https://adafru.it/IDI\)](https://adafru.it/IDI) in this guide.

Code

Use the Download button to get the code for the project.

```

1 // SPDX-FileCopyrightText: 2025 Adafruit Industries
2 //
3 // Based on code submitted via feedback to learn.adafruit.com
4 //
5 // SPDX-License-Identifier: Unlicense
6
7 #include <SPI.h>
8 #include <Adafruit_MLX90640.h>
9 #include <Adafruit_ILI9341.h>
10 #include <Adafruit_GFX.h>
11
12 // Different display select pin numbers depending on the board used
13 #ifdef ESP8266
14 #define STMPE_CS 16
15 #define TFT_CS 0
16 #define TFT_DC 15
17 #define SD_CS 2
18 #elif defined(ARDUINO_ADAFRUIT_FEATHER_ESP32C6)
19 #define STMPE_CS 6
20 #define TFT_CS 7
21 #define TFT_DC 8
22 #define SD_CS 5
23 #elif defined(ESP32) && !defined(ARDUINO_ADAFRUIT_FEATHER_ESP32S2)
24 #define STMPE_CS 32
25 #define TFT_CS 15
26 #define TFT_DC 33
27 #define SD_CS 14
28 #elif defined(TEENSYDUINO)
29 #define TFT_DC 10
30 #define TFT_CS 4
31 #define STMPE_CS 3
32 #define SD_CS 8
33 #elif defined(ARDUINO_STM32_FEATHER)
34 #define TFT_DC PB4
35 #define TFT_CS PA15
36 #define STMPE_CS PC7
37 #define SD_CS PC5
38 #elif defined(ARDUINO_NRF52832_FEATHER) /* BSP 0.6.5 and higher! */
39 #define TFT_DC 11
40 #define TFT_CS 31
41 #define STMPE_CS 30
42 #define SD_CS 27
43 #elif defined(ARDUINO_MAX32620FTHR) || defined(ARDUINO_MAX32630FTHR)
44 #define TFT_DC P5_4
45 #define TFT_CS P5_3
46 #define STMPE_CS P3_3
47 #define SD_CS P3_2
48 #else
49 // Anything else, defaults! This includes the Feather M0 Express
50 #define STMPE_CS 6
51 #define TFT_CS 9
52 #define TFT_DC 10
53 #define SD_CS 5
54 #endif
55
56 // Initialize the ILI9341 display

```

```

57 Adafruit_ILI9341 tft = Adafruit_ILI9341(TFT_CS, TFT_DC);
58
59 // Initialize thermal sensor
60 Adafruit_MLX90640 mlx;
61 float frame[32*24]; // buffer for full frame of temperatures
62
63 //low range of the sensor (this will be blue on the screen)
64 #define MINTEMP 20
65
66 //high range of the sensor (this will be red on the screen)
67 #define MAXTEMP 35
68
69 //the colors we will be using
70 const uint16_t camColors[] = {0x480F,
71 0x400F,0x400F,0x400F,0x4010,0x3810,0x3810,0x3810,0x3810,0x3010,0x3010,
72 0x3010,0x2810,0x2810,0x2810,0x2810,0x2010,0x2010,0x2010,0x1810,0x1810,
73 0x1811,0x1811,0x1011,0x1011,0x1011,0x0811,0x0811,0x0811,0x0011,0x0011,
74 0x0011,0x0011,0x0011,0x0031,0x0031,0x0051,0x0072,0x0072,0x0092,0x00B2,
75 0x00B2,0x00D2,0x00F2,0x00F2,0x0112,0x0132,0x0152,0x0172,0x0192,
76 0x0192,0x01B2,0x01D2,0x01F3,0x01F3,0x0213,0x0233,0x0253,0x0253,0x0273,
77 0x0293,0x02B3,0x02D3,0x02D3,0x02F3,0x0313,0x0333,0x0333,0x0353,0x0373,
78 0x0394,0x03B4,0x03D4,0x03D4,0x03F4,0x0414,0x0434,0x0454,0x0474,0x0474,
79 0x0494,0x04B4,0x04D4,0x04F4,0x0514,0x0534,0x0534,0x0554,0x0554,0x0574,
80 0x0574,0x0573,0x0573,0x0573,0x0573,0x0572,0x0572,0x0572,0x0571,0x0591,
81 0x0590,0x0590,0x058F,0x058F,0x058F,0x058E,0x05AE,0x05AE,0x05AD,0x05AD,
82 0x05AD,0x05AC,0x05AC,0x05AB,0x05CB,0x05CB,0x05CA,0x05CA,0x05CA,0x05C9,
83 0x05C9,0x05C8,0x05E8,0x05E8,0x05E7,0x05E7,0x05E6,0x05E6,0x05E6,0x05E5,
84 0x05E5,0x0604,0x0604,0x0604,0x0603,0x0603,0x0602,0x0602,0x0601,0x0621,
85 0x0621,0x0620,0x0620,0x0620,0x0620,0x0620,0x0E20,0x0E20,0x0E40,0x1640,0x1640,
86 0x1E40,0x1E40,0x2640,0x2640,0x2E40,0x2E60,0x3660,0x3660,0x3E60,0x3E60,
87 0x3E60,0x4660,0x4660,0x4E60,0x4E80,0x5680,0x5680,0x5E80,0x5E80,0x6680,
88 0x6680,0x6E80,0x6EA0,0x76A0,0x76A0,0x7EA0,0x7EA0,0x86A0,0x86A0,0x8EA0,
89 0x8EC0,0x96C0,0x96C0,0x9EC0,0x9EC0,0xA6C0,0xAEC0,0xAEC0,0xB6E0,0xB6E0,
90 0xBEE0,0xBEE0,0xC6E0,0xC6E0,0xC6E0,0xC6E0,0xD6E0,0xD700,0xDF00,0xDEE0,
91 0xDECO,0xDEA0,0xDE80,0xDE80,0xE660,0xE640,0xE620,0xE600,0xE5E0,0xE5C0,
92 0xE5A0,0xE580,0xE560,0xE540,0xE520,0xE500,0xE4E0,0xE4C0,0xE4A0,0xE480,
93 0xE460,0xEC40,0xEC20,0xEC00,0xEBE0,0xEBE0,0xEBE0,0xEBE0,0xEBE0,0xEBE0,0xEBE0,
94 0xEB20,0xEB00,0xEAE0,0xEAC0,0xEAA0,0xEA80,0xEA60,0xEA40,0xF220,0xF200,
95 0xF1E0,0xF1C0,0xF1A0,0xF180,0xF160,0xF140,0xF100,0xF0E0,0xF0C0,0xF0A0,
96 0xF080,0xF060,0xF040,0xF020,0xF800,};
97
98 uint16_t displayPixelWidth, displayPixelHeight;
99
100 void setup() {
101
102     Serial.begin(115200); // Initialize serial terminal
103     delay(10);
104
105     // Initialize and Setup TFT Display
106     tft.begin();
107     tft.setRotation(1);
108     tft.fillScreen(ILI9341_BLACK);
109     displayPixelWidth = tft.width() / 32;
110     displayPixelHeight = tft.height() / 32; //Keep pixels square
111
112     delay(100);

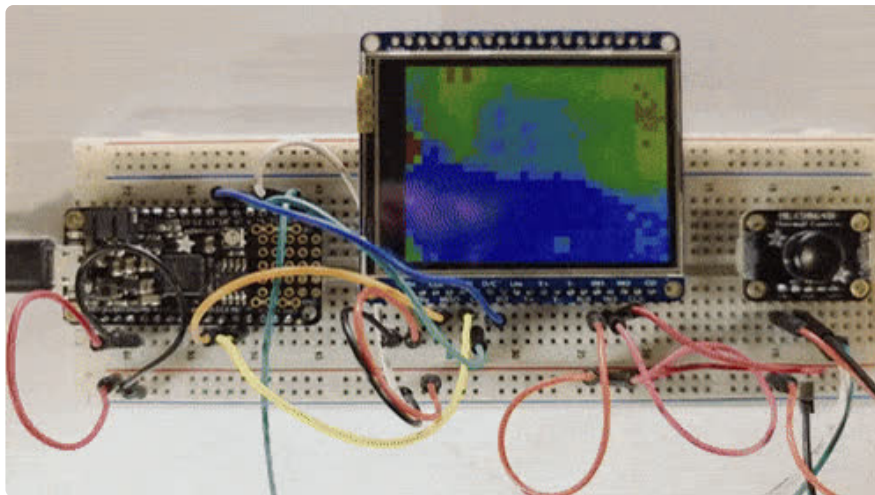
```

```

113
114 //Serial.println("Adafruit MLX90640 Thermal Camera");
115 //
116 // Read the MLX90640 via I2C (Arduino Wire)
117 if (! mlx.begin(MLX90640_I2CADDR_DEFAULT, &Wire)) {
118     // MLX90640 not found
119     tft.setCursor(0, 0);
120     tft.setTextColor(ILI9341_WHITE); tft.setTextSize(1);
121     tft.println("Thermal Sensor not found");
122     while(1) ; // Stop program with error
123 }
124 // Un-comment for debugging sensor set up
125 // Check sensor wiring if sensor isn't found and make sure it's an MLX90640
126 /*
127     Serial.println("Found Adafruit MLX90640");
128
129     Serial.print("Serial number: ");
130     Serial.print(mlx.serialNumber[0], HEX);
131     Serial.print(mlx.serialNumber[1], HEX);
132     Serial.println(mlx.serialNumber[2], HEX);
133 */
134
135 // Set up Thermal Sensor
136 mlx.setMode(MLX90640_CHESS);
137 mlx.setResolution(MLX90640_ADC_18BIT);
138 mlx.setRefreshRate(MLX90640_8_HZ); // COuld also try MLX90640_16_HZ
139 Wire.setClock(1000000); // max 1 MHz
140 }
141
142 // Main loop: Run the project continuously
143 void loop() {
144
145     if (mlx.getFrame(frame) != 0) { // Get thermal sensor data
146         Serial.println("Failed"); // Look in serial for failed messages if issues
147         return; // will loop again
148     }
149
150     // For every square on the screen, get the value and map it into a temperature color
151     for (uint8_t h=0; h<24; h++) {
152         for (uint8_t w=0; w<32; w++) {
153             float t = frame[h*32 + w];
154
155             t = min(t, MAXTEMP); // Constrain the temp value between MAXTEMP and MINTEMP
156             t = max(t, MINTEMP);
157
158             uint8_t colorIndex = map(t, MINTEMP, MAXTEMP, 0, 255); // Get square color
159             colorIndex = constrain(colorIndex, 0, 255); // and make sure it's valid
160
161             // Draw the pixels!
162             tft.fillRect(displayPixelWidth * w, displayPixelHeight * h,
163                 displayPixelHeight, displayPixelWidth,
164                 camColors[colorIndex]);
165         }
166     }
167 }

```

https://github.com/adafruit/Adafruit_MLX90640/blob/master/examples/MLX90640_ILI9341/Thermal.ino



You can change the line `tft.setRotation(1);` to one of the following values if you need to orient your display differently in relation to the sensor:

- | | |
|---|---|
| 1 | <code>tft.setRotation(0);</code> // Normal orientation |
| 2 | <code>tft.setRotation(1);</code> // Rotate 90° clockwise |
| 3 | <code>tft.setRotation(2);</code> // Upside down |
| 4 | <code>tft.setRotation(3);</code> // Rotate 270° clockwise |

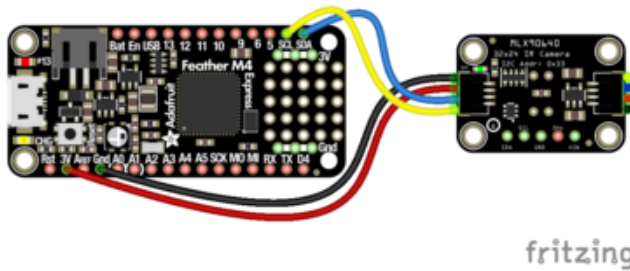
Python & CircuitPython

It's easy to use the MLX90640 sensor with Python and CircuitPython, and the [Adafruit CircuitPython MLX90640 \(https://adafru.it/IBv\)](https://adafru.it/IBv) module. This module allows you to easily write Python code that reads temperature using the thermal camera.

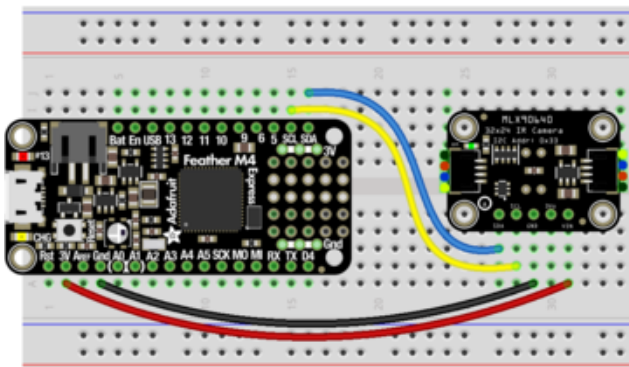
You can use this sensor with any CircuitPython microcontroller board or with a computer that has GPIO and Python [thanks to Adafruit_Blinka, our CircuitPython-for-Python compatibility library \(https://adafru.it/BSN\)](https://adafru.it/BSN).

CircuitPython Microcontroller Wiring

First wire up a MLX90640 to your board for an I2C connection, exactly as shown below. Here's an example of wiring a Feather M4 to the sensor with I2C:



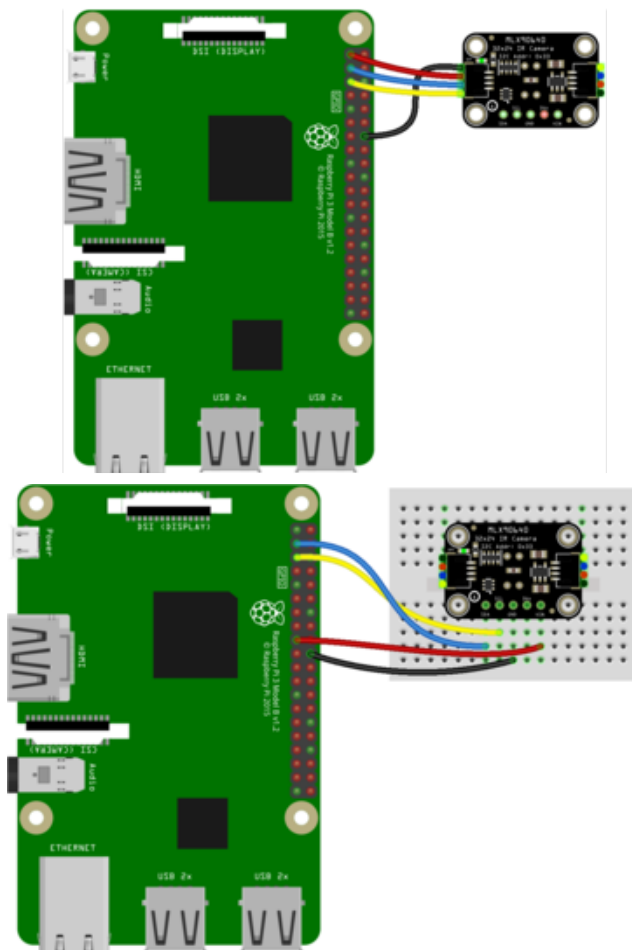
- Board 3V to sensor VIN (red wire)
- Board GND to sensor GND (black wire)
- Board SCL to sensor SCL (yellow wire)
- Board SDA to sensor SDA (blue wire)



Python Computer Wiring

Since there are dozens of Linux computers/boards you can use, we will show wiring for Raspberry Pi. For other platforms, [please visit the guide for CircuitPython on Linux to see whether your platform is supported](https://adafru.it/BSN) (<https://adafru.it/BSN>).

Here's the Raspberry Pi wired with I2C:



Pi 3V to sensor VCC (red wire)
 Pi GND to sensor GND (black wire)
 Pi SCL to sensor SCL (yellow wire)
 Pi SDA to sensor SDA (blue wire)

CircuitPython Installation of MLX90640 Library

You'll need to install the [Adafruit CircuitPython MLX90640](https://adafru.it/IBv) (<https://adafru.it/IBv>) library on your CircuitPython board.

First make sure you are running the [latest version of Adafruit CircuitPython](https://adafru.it/Amd) (<https://adafru.it/Amd>) for your board.

Next you'll need to install the necessary libraries to use the hardware--carefully follow the steps to find and install these libraries from [Adafruit's CircuitPython library bundle](https://adafru.it/ENC) (<https://adafru.it/ENC>). Our CircuitPython starter guide has [a great page on how to install the library bundle](https://adafru.it/ABU) (<https://adafru.it/ABU>).

You'll need to manually install the necessary libraries from the bundle:

- `adafruit_mlx90640.mpy`
- `adafruit_bus_device`

Before continuing, make sure your board's `lib` folder has the `adafruit_mlx90640.mpy`, and `adafruit_bus_device` files and folders copied over.

Python Installation of MLX90640 Library

You'll need to install the **Adafruit_Blinka** library that provides the CircuitPython support in Python. This may also require enabling I2C on your platform and verifying you are running Python 3. [Since each platform is a little different, and Linux changes often, please visit the CircuitPython on Linux guide to get your computer ready \(https://adafru.it/BSN\)](https://adafru.it/BSN)!

Once that's done, from your command line run the following command:

- `pip3 install adafruit-circuitpython-mlx90640`

If your default Python is version 3 you may need to run 'pip' instead. Just make sure you aren't trying to use CircuitPython on Python 2.x, it isn't supported!

CircuitPython & Python Usage

To demonstrate the usage of the sensor we'll run the `mlx90640_simpletest.py` program which prints the temperatures or shows them as ASCII. As this example is too complicated to run from the REPL, you'll save the following code to your board as `code.py` and connect to the serial console to see the output.

```
1  # SPDX-FileCopyrightText: 2021 ladyada for Adafruit Industries
2  # SPDX-License-Identifier: MIT
3
4  import time
5
6  import board
7  import busio
8
9  import adafruit_mlx90640
10
11  PRINT_TEMPERATURES = False
12  PRINT_ASCIIART = True
13
14  i2c = busio.I2C(board.SCL, board.SDA, frequency=800000)
15  # i2c = board.STEMMA_I2C() # For using the built-in STEMMA QT connector on a microcontroller
16
17  mlx = adafruit_mlx90640.MLX90640(i2c)
18  print("MLX addr detected on I2C")
19  print([hex(i) for i in mlx.serial_number])
20
21  mlx.refresh_rate = adafruit_mlx90640.RefreshRate.REFRESH_2_HZ
22
23  frame = [0] * 768
24  while True:
25      stamp = time.monotonic()
26      try:
27          mlx.getFrame(frame)
```

```

28 except ValueError:
29     # these happen, no biggie - retry
30     continue
31 print("Read 2 frames in %0.2f s" % (time.monotonic() - stamp))
32 for h in range(24):
33     for w in range(32):
34         t = frame[h * 32 + w]
35         if PRINT_TEMPERATURES:
36             print(f"{t:0.1f}, ", end="")
37         if PRINT_ART:
38             c = "&"
39             if t < 20:
40                 c = " "
41             elif t < 23:
42                 c = "."
43             elif t < 25:
44                 c = "_"
45             elif t < 27:
46                 c = "*"
47             elif t < 29:
48                 c = "+"
49             elif t < 31:
50                 c = "x"
51             elif t < 33:
52                 c = "%"
53             elif t < 35:
54                 c = "#"
55             elif t < 37:
56                 c = "X"
57             print(c, end="")
58     print()
59 print()

```

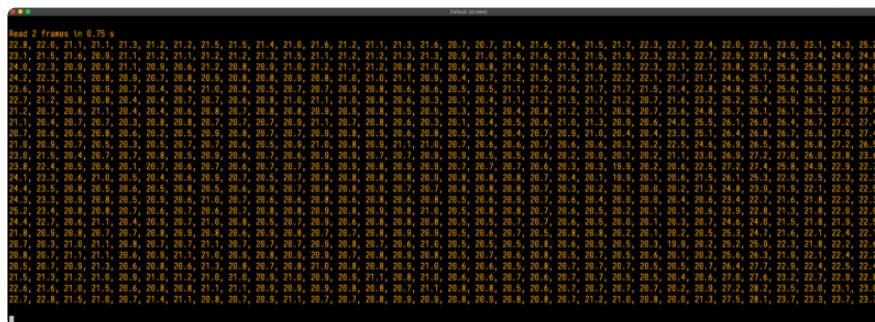
https://github.com/adafruit/Adafruit_CircuitPython_MLX90640/blob/main/examples/mlx90640_simpletest.py



If you change the values for the variables at the top of the program, you can switch from printing out an ASCII image to printing out the temperatures in a grid. Change `PRINT_TEMPERATURES` to `True` and `PRINT_ASCIIART` to `False`, so that the two lines are as follows:

1	<code>PRINT_TEMPERATURES = True</code>
2	<code>PRINT_ASCIIART = False</code>

Connect to the serial console to see the temperatures printed out in a grid. Fitting them all in a proper grid involved making my terminal window significantly larger than the default size.



That's all there is to using the MLX90640 with CircuitPython!

Python Docs

[Python Docs \(https://adafru.it/lxD\)](https://adafru.it/lxD)

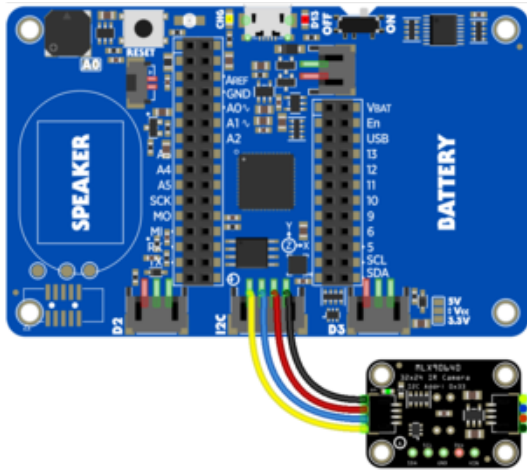
CircuitPython Thermal Camera

The MLX90640 comes with STEMMA QT/QWIIC connectors which makes it super simple to plug into projects with no soldering needed. Adafruit sells a [STEMMA to STEMMA QT cable \(http://adafru.it/4424\)](http://adafru.it/4424) that allows you to plug this breakout into the STEMMA connector found on a number of Adafruit microcontroller boards, including PyBadge and PyGamer. The following example uses the PyBadge or PyGamer and the MLX90640 to create a super easy-to-assemble thermal camera with a display!

CircuitPython Microcontroller Wiring

First wire up a MLX90640 to your PyBadge or PyGamer exactly as shown below.

Here's an example of wiring a PyBadge to the sensor with I2C using the [STEMMA to STEMMA QT cable](http://adafru.it/4424) (<http://adafru.it/4424>):



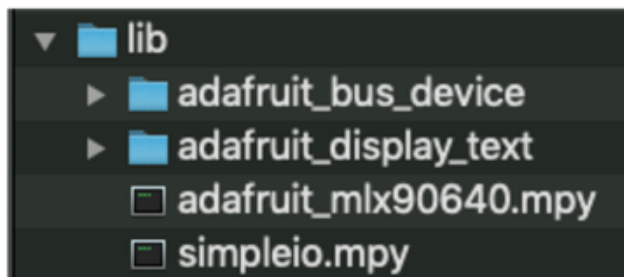
Plug the larger end (STEMMA/Grove) of the cable into the PyBadge/PyGamer. Plug the smaller end (STEMMA QT) of the cable into the MLX90640.

CircuitPython Installation of Additional Libraries

On the previous page, you installed the [Adafruit CircuitPython MLX90640](https://adafru.it/IBv) (<https://adafru.it/IBv>) library on your CircuitPython board.

First make sure you are running the [latest version of Adafruit CircuitPython](https://adafru.it/Amd) (<https://adafru.it/Amd>) for your board.

Next you'll need to install the necessary libraries to use this example -- carefully follow the steps to find and install these libraries from [Adafruit's CircuitPython library bundle](https://adafru.it/ENC) (<https://adafru.it/ENC>). Our CircuitPython starter guide has [a great page on how to install the library bundle](https://adafru.it/ABU) (<https://adafru.it/ABU>).



You'll need to manually install **two additional** libraries from the bundle:

`adafruit_display_text`
`simpleio.mpy`

Before continuing, make sure your board's lib folder has the `adafruit_mlx90640.mpy`, `adafruit_bus_device`, `adafruit_display_text` and `simpleio.mpy` files and folders copied over.

CircuitPython PyBadge/PyGamer Thermal Camera

Save the following code to your PyBadge or PyGamer as **code.py**.

```
1  # SPDX-FileCopyrightText: 2021 ladyada for Adafruit Industries
2  # SPDX-License-Identifier: MIT
3
4  import time
5
6  import board
7  import busio
8  import displayio
9  import terminalio
10 from adafruit_display_text.label import Label
11 from simpleio import map_range
12
13 import adafruit_mlx90640
14
15 number_of_colors = 64 # Number of color in the gradient
16 last_color = number_of_colors - 1 # Last color in palette
17 palette = displayio.Palette(number_of_colors) # Palette with all our colors
18
19 ## Heatmap code inspired from: http://www.andrewnoske.com/wiki/Code\_-\_heatmaps\_and\_color\_gradients
20
21 color_A = [
22     [0, 0, 0],
23     [0, 0, 255],
24     [0, 255, 255],
25     [0, 255, 0],
26     [255, 255, 0],
27     [255, 0, 0],
28     [255, 255, 255],
29 ]
30 color_B = [[0, 0, 255], [0, 255, 255], [0, 255, 0], [255, 255, 0], [255, 0, 0]]
31 color_C = [[0, 0, 0], [255, 255, 255]]
32 color_D = [[0, 0, 255], [255, 0, 0]]
33
34 color = color_B
35 NUM_COLORS = len(color)
36
37
38 def MakeHeatMapColor():
39     for c in range(number_of_colors):
40         value = c * (NUM_COLORS - 1) / last_color
41         idx1 = int(value) # Our desired color will be after this index.
42         if idx1 == value: # This is the corner case
43             red = color[idx1][0]
44             green = color[idx1][1]
45             blue = color[idx1][2]
46         else:
47             idx2 = idx1 + 1 # ... and before this index (inclusive).
48             fractBetween = value - idx1 # Distance between the two indexes (0-1).
49             red = int(round((color[idx2][0] - color[idx1][0]) * fractBetween + color[idx1][0]))
50             green = int(round((color[idx2][1] - color[idx1][1]) * fractBetween + color[idx1][1]))
51             blue = int(round((color[idx2][2] - color[idx1][2]) * fractBetween + color[idx1][2]))
52             palette[c] = (0x010000 * red) + (0x000100 * green) + (0x000001 * blue)
```

```

53
54
55 MakeHeatMapColor()
56
57 # Bitmap for colour coded thermal value
58 image_bitmap = displayio.Bitmap(32, 24, number_of_colors)
59 # Create a TileGrid using the Bitmap and Palette
60 image_tile = displayio.TileGrid(image_bitmap, pixel_shader=palette)
61 # Create a Group that scale 32*24 to 128*96
62 image_group = displayio.Group(scale=4)
63 image_group.append(image_tile)
64
65 scale_bitmap = displayio.Bitmap(number_of_colors, 1, number_of_colors)
66 # Create a Group Scale must be 128 divided by number_of_colors
67 scale_group = displayio.Group(scale=2)
68 scale_tile = displayio.TileGrid(scale_bitmap, pixel_shader=palette, x=0, y=60)
69 scale_group.append(scale_tile)
70
71 for i in range(number_of_colors):
72     scale_bitmap[i, 0] = i # Fill the scale with the palette gradian
73
74 # Create the super Group
75 group = displayio.Group()
76
77 min_label = Label(terminalio.FONT, color=palette[0], x=0, y=110)
78 max_label = Label(terminalio.FONT, color=palette[last_color], x=80, y=110)
79
80 # Add all the sub-group to the SuperGroup
81 group.append(image_group)
82 group.append(scale_group)
83 group.append(min_label)
84 group.append(max_label)
85
86 # Add the SuperGroup to the Display
87 board.DISPLAY.root_group = group
88
89 min_t = 20 # Initial minimum temperature range, before auto scale
90 max_t = 37 # Initial maximum temperature range, before auto scale
91
92 i2c = busio.I2C(board.SCL, board.SDA, frequency=800000)
93 # i2c = board.STEMMA_I2C() # For using the built-in STEMMMA QT connector on a microcontroller
94
95 mlx = adafruit_mlx90640.MLX90640(i2c)
96 print("MLX addr detected on I2C")
97 print([hex(i) for i in mlx.serial_number])
98
99 # mlx.refresh_rate = adafruit_mlx90640.RefreshRate.REFRESH_2_HZ
100 mlx.refresh_rate = adafruit_mlx90640.RefreshRate.REFRESH_4_HZ
101
102 frame = [0] * 768
103
104 while True:
105     stamp = time.monotonic()
106     try:
107         mlx.getFrame(frame)
108     except ValueError:

```

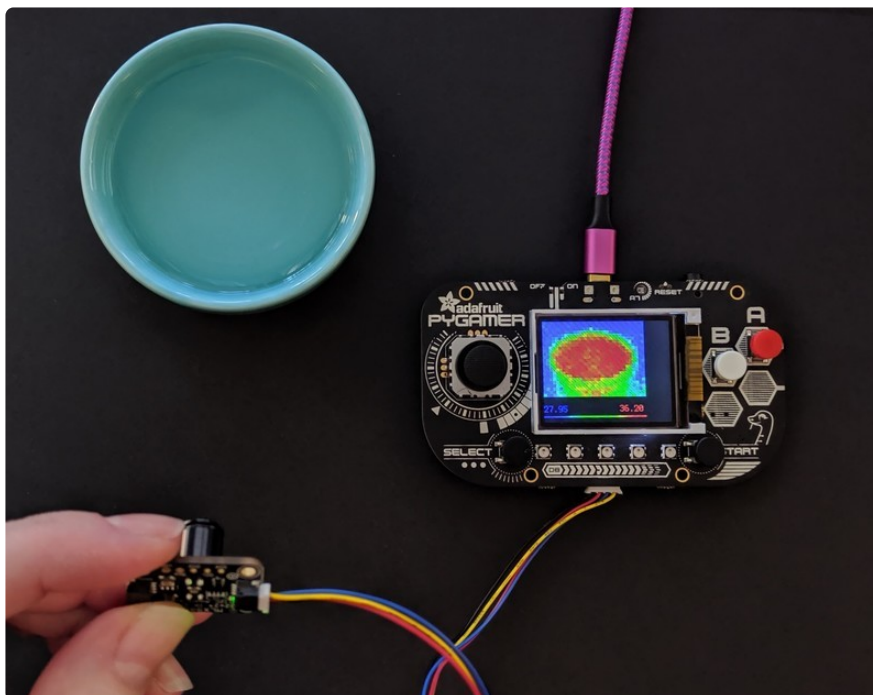
```

109     # these happen, no biggie - retry
110     continue
111
112     # print("Time for data aquisition: %0.2f s" % (time.monotonic()-stamp))
113
114     mini = frame[0] # Define a min temperature of current image
115     maxi = frame[0] # Define a max temperature of current image
116
117     for h in range(24):
118         for w in range(32):
119             t = frame[h * 32 + w]
120             maxi = max(maxi, t)
121             mini = min(mini, t)
122             image_bitmap[w, (23 - h)] = int(map_range(t, min_t, max_t, 0, last_color))
123
124     min_label.text = f"{min_t:0.2f}"
125
126     max_string = f"{max_t:0.2f}"
127     max_label.x = 120 - (5 * len(max_string)) # Tricky calculation to left align
128     max_label.text = max_string
129
130     min_t = mini # Automatically change the color scale
131     max_t = maxi
132     # print((mini, maxi)) # Use this line to display min and max graph in Mu
133     # print("Total time for aquisition and display %0.2f s" % (time.monotonic()-stamp))

```

https://github.com/adafruit/Adafruit_CircuitPython_MLX90640/blob/main/examples/mlx90640_pygamer.py

Now point the thermal camera at various objects to see a heat map displayed on your board!

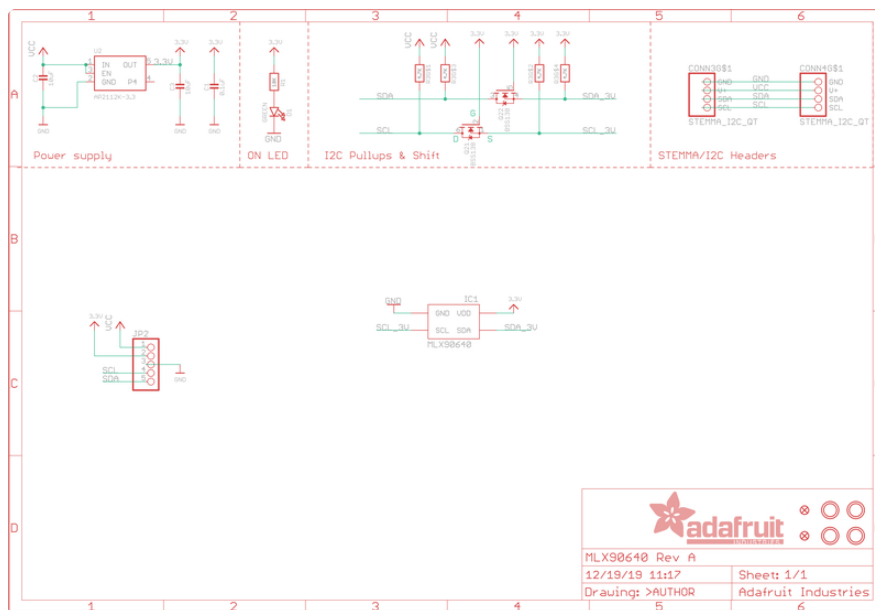


Downloads

Files

- [MLX90640 datasheet \(https://adafru.it/IAR\)](https://adafru.it/IAR)
- [EagleCAD files on GitHub \(https://adafru.it/IAS\)](https://adafru.it/IAS)
- [3D model files on GitHub \(https://adafru.it/K5E\)](https://adafru.it/K5E)
- [Fritzing object in Adafruit Fritzing Library \(https://adafru.it/IAT\)](https://adafru.it/IAT)

Schematic



Fab Print

